

Лабораторная работа №3. Диалог, формы и состояние сессии

Что вы сделаете

Вы добавите страницу `/services/chat/`, на которой пользователь обменивается короткими сообщениями с локальной моделью. Django будет хранить реплики в правильном порядке, отделять диалоги разных браузеров и очищать только текущий чат.

Главные темы — обычная `Form`, два POST-действия, работа с последовательностью записей и целостность данных. Код генерации уже находится в `services.py`; устройство нейросети не требуется изучать.

1. Исходный проект

Продолжите ЛР2 либо распакуйте `starter.zip` из каталога ЛР3. Starter — проверенное состояние после ЛР2 без `.venv`, `db.sqlite3`, `media` и кеша моделей. Команды выполняйте рядом с `manage.py`.

Если окружение создаётся заново, выполните команды из ЛР1, затем установите зависимости проекта. Дополнение для этой работы одинаково на Windows, Linux и macOS:

```
python -m pip install sentencepiece==0.2.2 protobuf==6.33.6
```

PyTorch и Transformers уже должны быть установлены после ЛР2. Быстрая проверка:

```
python -c "import torch, transformers, sentencepiece; print('Пакеты установлены')"
```

2. Готовые файлы

Распакуйте содержимое `готовые_файлы.zip` из каталога ЛР3 прямо в каталог с `manage.py`. Появятся приложение `dialog_bot`, шаблон `templates/dialog_bot/index.html` и обновлённая команда подготовки моделей.

Не запускайте `startapp`: все необходимые файлы уже созданы. Их назначение:

- `models.py` — одна реплика диалога;
- `forms.py` — ввод сообщения;
- `views.py` — отправка и очистка;
- `urls.py` — два адреса;
- `services.py` — готовая обёртка локальной модели;
- `tests.py` — автоматические проверки.

3. Подключение приложения

Добавьте в `INSTALLED_APPS` файла `config/settings.py`:

```
"dialog_bot",
```

Добавьте в `urlpatterns` файла `config/urls.py`:

```
path("services/chat/", include("dialog_bot.urls")),
```

В `templates/base.html` после ссылки «Эмоции» добавьте:

```
<li><a href="{% url 'dialog_bot:index' %}"
{% if active_page == 'chat' %}aria-current="page"{% endif %}>Диалог</a></li>
```

В карточке «Диалоговый бот» файла `core/views.py` замените `"url_name": None` на:

```
"url_name": "dialog_bot:index",
```

Проверьте конфигурацию:

```
python manage.py check
```

4. Модель сообщения и миграция

Откройте `dialog_bot/models.py`. `ChatMessage` содержит ключ сессии, роль, текст и дату. Допустимые роли описаны через `TextChoices`: `user` и `assistant`. Выбор ограничивает случайные значения и даёт русские подписи.

Порядок по умолчанию задан как `created_at`, поэтому старые реплики идут раньше новых. Создайте таблицу:

```
python manage.py makemigrations dialog_bot
python manage.py migrate
```

Если изменить модель позже, эти две команды нужно повторить.

5. Почему здесь обычная Form

Откройте `dialog_bot/forms.py`. `ChatForm` наследуется от `forms.Form`, а не `ModelForm`. Пользователь вводит только временное поле `message`. Сервер сам создаёт две разные записи: сообщение пользователя и ответ.

Форма ограничивает ввод 300 символами. Модель допускает 1000, потому что автоматически полученный ответ иногда длиннее пользовательского текста. Поля `role` и `session_key` отсутствуют в форме: браузеру нельзя доверять выбор автора или чужой сессии.

6. Маршруты и два действия

В `dialog_bot/urls.py` определены:

```
path("", views.index, name="index")
path("clear/", views.clear, name="clear")
```

С учётом корневого `include` это адреса `/services/chat/` и `/services/chat/clear/`.

`index` принимает GET и POST. `clear` помечен `@require_POST`, потому что очистка изменяет данные. Обычная ссылка отправляет GET и не должна удалять историю. В шаблоне очистка поэтому оформлена отдельной формой с `{% csrf_token %}`.

7. Разбор отправки сообщения

Откройте `dialog_bot/views.py` и проследите успешный POST:

1. `ChatForm(request.POST)` получает данные.

2. После `is_valid()` берётся `cleaned_data["message"]`.
3. Из БД выбираются три последние реплики только текущей сессии.
4. К ним добавляется новое сообщение; готовый сервис получает максимум четыре реплики.
5. После получения ответа две записи сохраняются внутри `transaction.atomic()`.
6. View делает `redirect` обратно к разговору.

`transaction.atomic()` означает: либо сохраняются обе реплики, либо ни одна. Это предотвращает половину диалога при ошибке базы.

Историю нельзя хранить в глобальном списке Python. Один сервер обслуживает разных пользователей, список смешает их сообщения и исчезнет после перезапуска. Здесь состояние хранится в БД и всегда фильтруется по `session_key`.

8. Очистка и вывод истории

Функция `clear` получает текущий ключ и выполняет:

```
ChatMessage.objects.filter(session_key=session_key).delete()
```

Фильтр обязателен: вызов `ChatMessage.objects.all().delete()` очистил бы чаты всех пользователей.

Для вывода `view` выбирает до 20 последних записей, затем разворачивает список, чтобы на странице они шли от старых к новым. В шаблоне классы для пользователя и ассистента различают расположение сообщений, а текстовые подписи ролей не заставляют угадывать смысл цвета.

9. Готовый сервис генерации

`dialog_bot/services.py` объединяет последние реплики, ограничивает длину входа и ответа и вызывает небольшую локальную модель. Для лабораторной рассматривайте его как функцию:

```
список строк → строка ответа
```

Модель развлекательная: она может повторяться, ошибаться и не является источником достоверных советов. Это честно написано рядом с формой.

При желании скачайте файлы заранее:

```
python manage.py prepare_models --service chat
```

На первом запуске нужен интернет. Дальнейшая работа выполняется локально.

10. Запуск и проверка

```
python manage.py runserver
```

Откройте `<http://127.0.0.1:8000/services/chat/>` и проверьте:

1. До отправки видно понятное пустое состояние.
2. После сообщения появляются две различимые реплики.
3. Обновление страницы не создаёт повторные записи.
4. «Очистить историю» возвращает пустое состояние.

5. В приватном окне создаётся независимый чат; его очистка не влияет на обычное окно.

11. Автоматические тесты

```
python manage.py check
python manage.py makemigrations --check --dry-run
python manage.py test
```

В тестах реальная генерация заменена заглушкой. Проверяются две создаваемые записи, порядок ролей, redirect, ограничение контекста, длина формы, POST-очистка и изоляция сессий.

12. Три полезные UX-проверки

1. Попросите одноклассника определить, где его сообщения, а где ответы. Подписи и расположение должны быть понятны без объяснения цветов.
2. Отправьте сообщение и оцените обратную связь во время ожидания: кнопка не должна провоцировать повторные нажатия.
3. Очистите чат. После действия должно быть очевидно, что история удалена и можно начать новый разговор.

Запишите одно наблюдение и результат. Не требуется обязательно находить недостаток.

13. Если что-то не работает

- `No module named sentencepiece` — активируйте нужное окружение и повторите установку из пункта 1.
- `NoReverseMatch: dialog_bot` — проверьте `INSTALLED_APPS`, корневой URL и `app_name` в `dialog_bot/urls.py`.
- `no such table: dialog_bot_chatmessage` — выполните миграции из пункта 4.
- После обновления появляются дубликаты — успешный POST должен завершаться `redirect`.
- Сообщения двух окон смешиваются — каждый запрос к модели должен содержать `session_key=session_key`.
- Очистка возвращает 405 при вводе адреса вручную — это правильно: маршрут принимает только POST; используйте кнопку.
- Ответ долго не появляется первый раз — идёт скачивание модели; проверьте терминал и интернет.

14. Контрольные вопросы

1. Почему для сообщения используется `Form`, а не `ModelForm`?
2. Почему роль и ключ сессии назначает сервер?
3. Зачем очистка принимает POST, а не GET?
4. Что даёт `transaction.atomic()` при сохранении двух сообщений?
5. Почему глобальный список Python не подходит для истории?
6. Как запрос к БД отделяет диалоги разных браузеров?

15. Критерии приёмки и отчёт

Чат доступен из меню; после отправки хранит две реплики в правильном порядке; обновление не дублирует их; очистка затрагивает только текущую сессию; пустое состояние и роли понятны; тесты проходят.

В отчёт DOCX/PDF включите титульные данные, цель, добавленные файлы, скриншоты пустого и заполненного чата, объяснение отправки и очистки, результат UX-проверок и ответы. На защите запустите чат и покажите `models.py`, `forms.py`, оба `view` и маршруты.